# AISE 4010

# Lecture 2:
# Time Series Forecasting
# (Statistical Models)

September 16, 2026

# Outline

- Different time series pre-processing techniques
- Autoregressive forecasting models
- Evaluating and backtesting forecasts

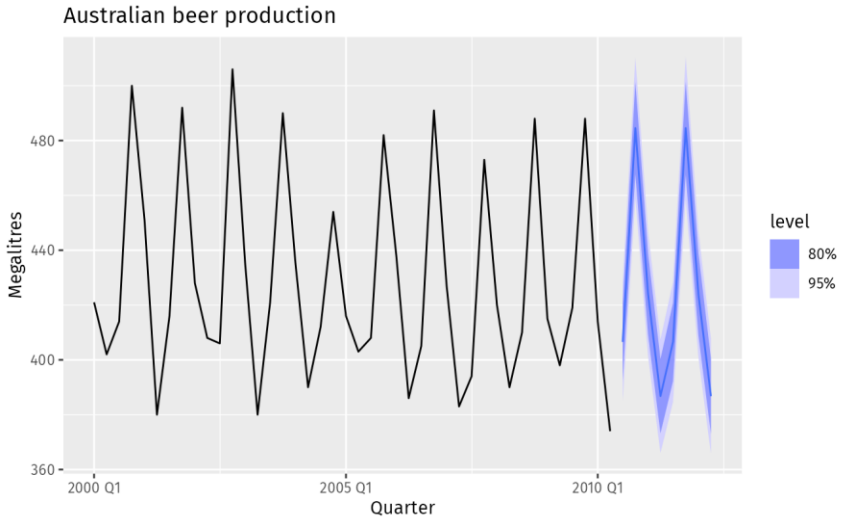
# Time Series Prediction/Forecasting

- In practice, most of the time series represent stochastic characteristics, and clearly the future value of the time series, cannot be precisely predicted.

- There is always an error associated with the prediction

- To predict time series value at a certain point of time, there are fundamentally two different strategies:
  - machine learning-based (learn variations)
  - model-based/statistical strategies (system identification using a linear/nonlinear model), such as:

    - ARIMA
    - SARIMA



# 2.1: Cleaning Your Data

- Missing Data
- Changing frequency
- Removing Non-stationary

# Handling Missing Values in Time Series

- Missing data is surprisingly common. It is necessary to find the time values that are missing from the time series.
- Missing data is even more common in time series data because the burden of longitudinal sampling is particularly heavy
- Some methods allow for missing values without any problems, but some other modelling functions do not handle missing values!

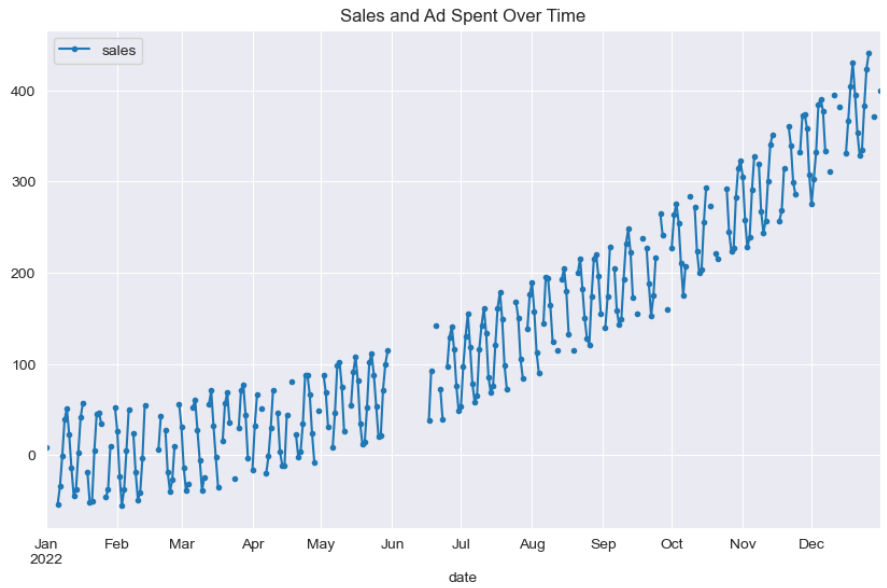
Ex. In sales data:

- missing values occur on public holidays when the store is closed
- someone may have forgotten to record the sales figures
- data recording device may have malfunctioned

```
# Print the percentage of missing values
print('% missing data in sales: ', 100*df['sales'].isnull().sum()/len(df))
```

| date | sales | ad_spent |
|---|---|---|
| 2022-01-01 | 8.830262 | 6.593897 |
| 2022-01-02 | NaN | 2.503652 |
| 2022-01-03 | NaN | 0.000000 |
| 2022-01-04 | NaN | 0.000000 |
| 2022-01-05 | NaN | 0.000000 |

```
% missing data in sales:  24.10958904109589
```



# Types of Missing Values and Handling Missing Values

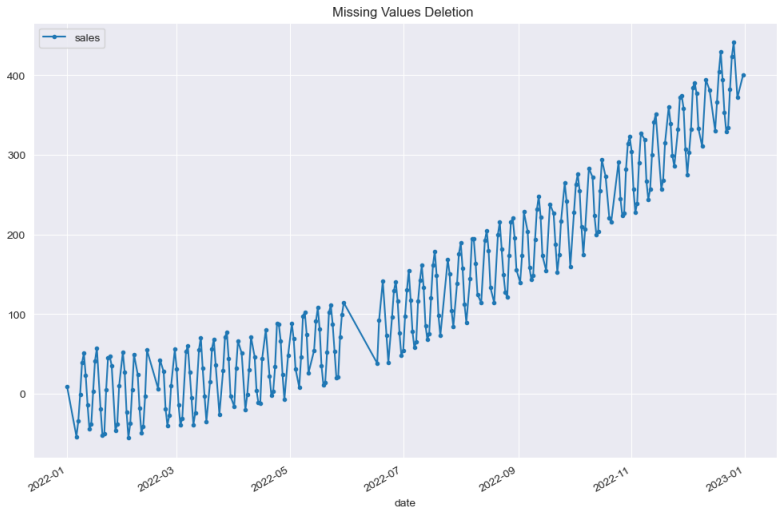
- MCAR (Missing Completely at Random): the probability of a value being missing is unrelated to any variable, observed or not. This is the only case in which simply deleting or mean-imputing the affected rows is unbiased.
- MAR (Missing at Random): Variable has missing values that are randomly distributed. However, they are related to some other variables' values.
- MNAR (Missing not at Random/Systematic Missing Data): The missingness of data is related to events or factors which are not measured by the researcher.

- The most common methods to address missing data in time series are:
  - Imputation: when we fill in missing data based on observations about the entire dataset.
  - Interpolation: when we use neighboring data points to estimate the missing value. Interpolation can also be a form of imputation.
  - Deletion of affected time periods: when we choose not to use time periods that have missing data at all. It will result in less data for your model!!

# Handling Missing Values

- Deletion (not recommended)
  - This strategy eliminates any rows that contain missing values. Straightforward to execute but this approach may result in valuable information loss.
  - Not suitable for large amounts of missing data.

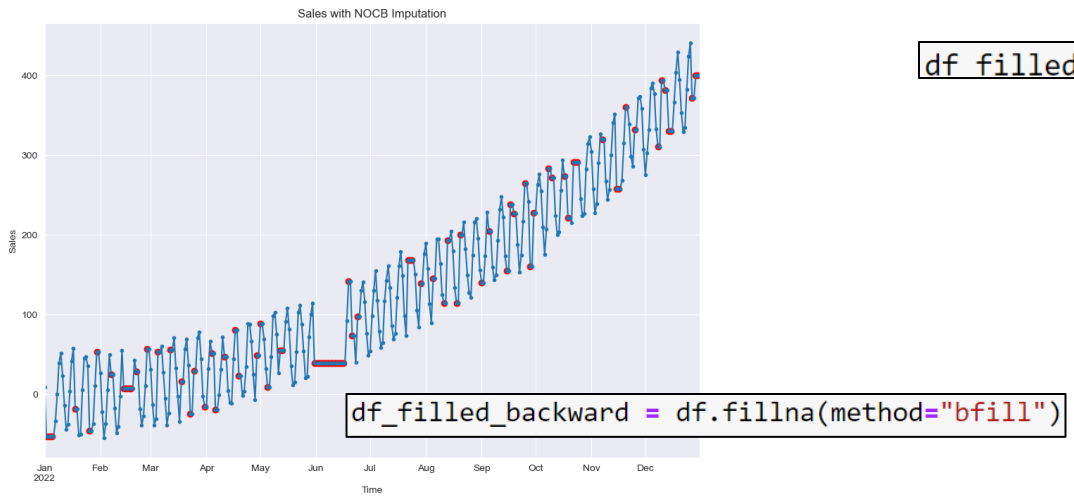
```
# Remove rows with missing values
df_deletion.dropna(inplace=True)
```

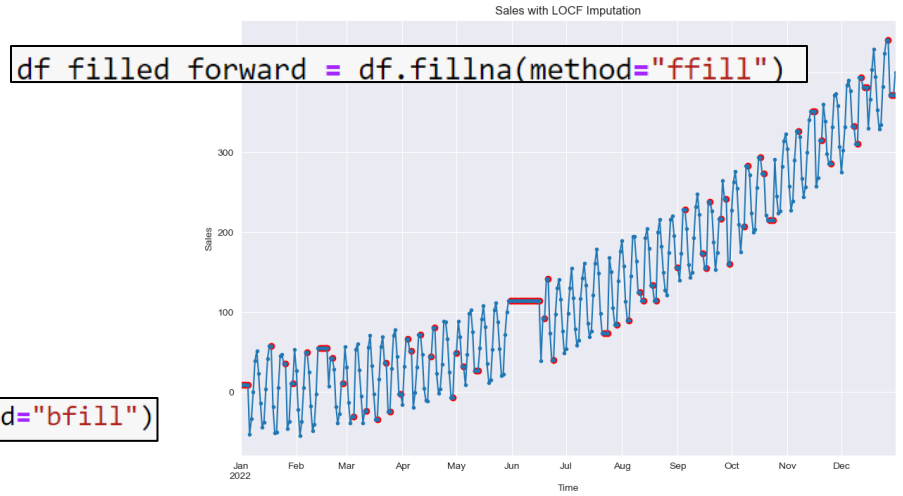



- Statistical Imputation (not recommended)
  - In this method, the missing values are replaced with the mean, median, or mode of the variable..
  - One of the most common methods.
  - Use it when the data is missing completely at random and the percentage of missing values is not huge

# Handling Missing Values

- Forward/Backward-Fill or Last/Next Observation Carried Forward/Backward (LOCF/NOCB)
  - One of the simplest ways to fill in missing values
  - When there is a trend or pattern that continues from one time point to the next
  - Fill missing values using the most recent succeeding or preceding value, respectively.
  - We don't consider any relationship between the data values
  - Useful when dealing with time series data that exhibits a relatively stable trend or when missing values occur in consecutive intervals
  - Note: you should only do NOCB/bfill when you are not looking to predict the future from the data and when, from domain knowledge, it makes more sense to fill in data backward rather than forward in time.

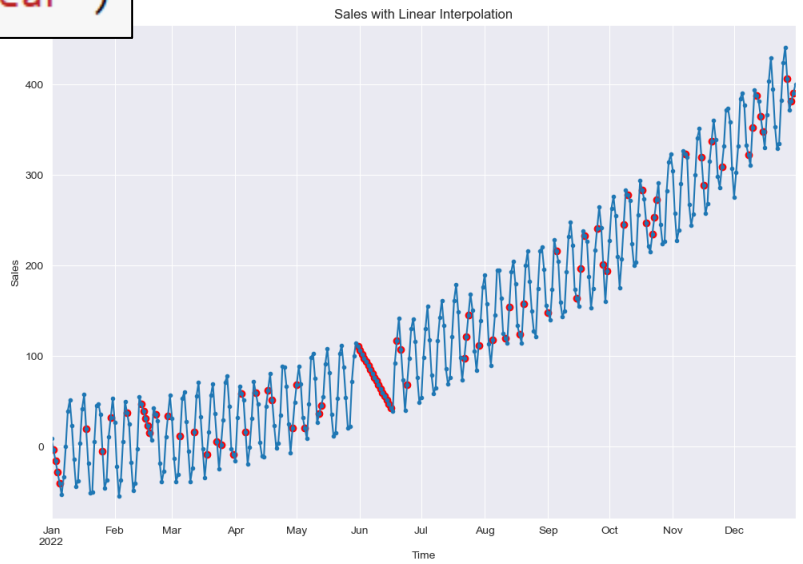


# Handling Missing Values

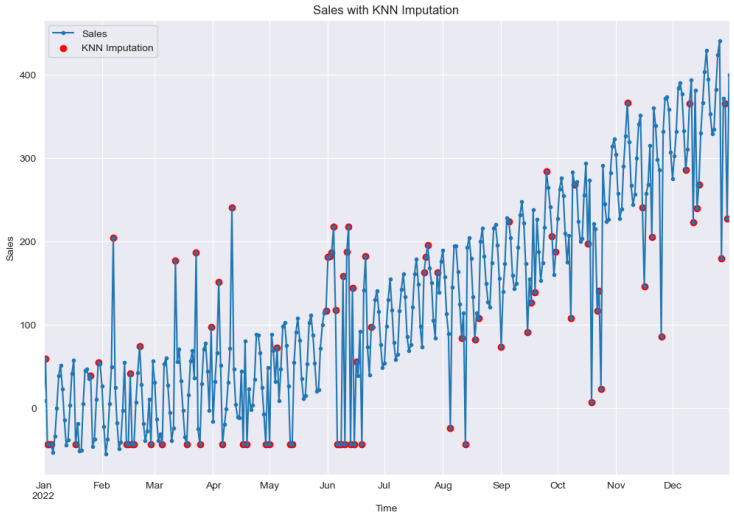
- Linear Interpolation
  - Estimating values between two known data points (assuming a straight line between data points)
  - Simple and fast
  - We need to consider a linear relationship between the known data points
  - It is advised not to use this method of imputing data when the underlying relationship is nonlinear, if the data has seasonal pattern or there are significant fluctuations or irregularities in the data (because it will almost distort the seasonal pattern).

```
# Apply the linear interpolation method
df_imputed = df.interpolate(method ='linear')
```



# Handling Missing Values

- Regression imputation
  - Is a method of imputing missing values by using the relationships between the data. In this method, a regression model is used to predict the missing values based on other data.
  - Can be used when the data is numeric and there is a strong correlation between the variable with missing values and other variables.
- K-Nearest Neighbors (KNN) Imputation
  - Finding the K-nearest neighbors to the observation with missing data and then imputing them based on the non-missing values in the neighbors.
  - Can be used when the data are missing at random
  - Simple and easy to implement
  - Not suitable for high dimensional data




- Seasonal-Trend Decomposition
  - Can capture complex seasonal patterns that other imputation methods might miss.
  - Not appropriate if there is a high level of noise in the data.
  - Algorithm:
    - Time series decomposition
    - Missing value identification
    - Impute Seasonal Component
    - Impute Trend Component: If still any missing values fill using for example linear interpolation, LOCF, or regression-based imputation
    - Reconstruct the Time Series.

# 2.2: Upsampling and Downsampling

# Resampling: Changing the frequency of time series

- Resampling refers to change the frequency of time series observations.
- Resampling includes upsampling and downsampling, for increasing or decreasing the timestamp frequency, respectively (increase or reduce frequency of your data, respectively).
- The reason for resampling:
  - Matching frequencies: Aligning time series data from different sources or for different analyses that require a consistent frequency.
  - Data exploration: Examining data behavior at different resolutions to gain insights.
  - Computational efficiency: downsampling can reduce dataset size and computational load for analysis or model training.
  - Modelling requirements: Some models may require data at a specific frequency.

# Upsampling/Downsampling

- **Upsampling**: e.g., converting hourly data to minute-by-minute data. New data points are introduced between existing observations.
- Requires an interpolation method to estimate the values for these new, higher-frequency points.
- Common interpolation techniques include linear interpolation, cubic interpolation

```
print("Original daily sampled data:\n", df.head())
# Upsample to hourly data and interpolate linearly
df_hourly_interpolated = df.resample('H').mean().interpolate(method='linear')
print("\nUpsampled (Hourly with Linear Interpolation):\n", df_hourly_interpolated.head(10))
```

```
Original daily sampled data:
                Value
2023-01-01  0.235834
2023-01-02  0.620500
2023-01-03  0.639622
2023-01-04  0.948540
2023-01-05  0.778276
```

```
Upsampled (Hourly with Linear Interpolation):
                      Value
2023-01-01 00:00:00  0.235834
2023-01-01 01:00:00  0.251862
2023-01-01 02:00:00  0.267890
2023-01-01 03:00:00  0.283917
2023-01-01 04:00:00  0.299945
2023-01-01 05:00:00  0.315973
2023-01-01 06:00:00  0.332001
2023-01-01 07:00:00  0.348028
2023-01-01 08:00:00  0.364056
2023-01-01 09:00:00  0.380084
```

- **Downsampling**: e.g., converting daily data to monthly data.
- Multiple observations from higher frequency aggregated into a single observation at lower frequency.
- Typically involves an aggregation function like the mean, sum, median, or max of values within new lower-frequency interval.

```
print("Original daily sampled data:\n", df.head())
# Downsample to weekly data, taking the mean
df_weekly = df.resample('W').mean()
print("Downsampled (Weekly Mean):\n", df_weekly.head())
```

```
Original daily sampled data:
                Value
2023-01-01  0.921719
2023-01-02  0.805264
2023-01-03  0.723941
2023-01-04  0.559174
2023-01-05  0.922299
```

```
Downsampled (Weekly Mean):
                Value
2023-01-01  0.921719
2023-01-08  0.744408
2023-01-15  0.399049
2023-01-22  0.437106
2023-01-29  0.411893
```

# 2.3: Removing Non-stationary Effect

# Recap: Stationary Analysis

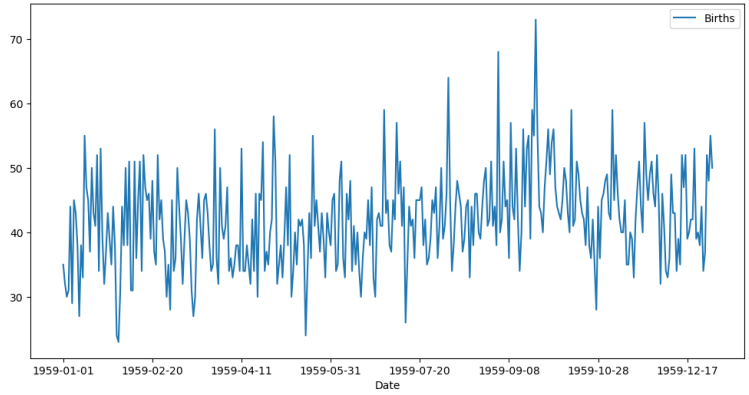
When time series is stationary, it can be easier to model. Statistical modeling methods assume or require time series to be stationary to be effective.

- Time series are stationary if statistics are consistent over time, like the mean, the variance, or the autocorrelation of the observations.

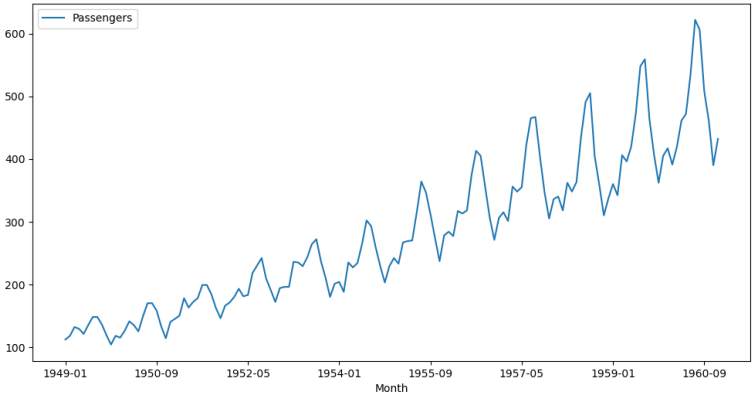
$E[y_t] = \mu$ constant for all $t$

$\mathrm{Var}(y_t) = \sigma^2$ constant

$\mathrm{Cov}(y_t, y_{t-k})$ depends **only on lag** $k$, not on the calendar time $t$

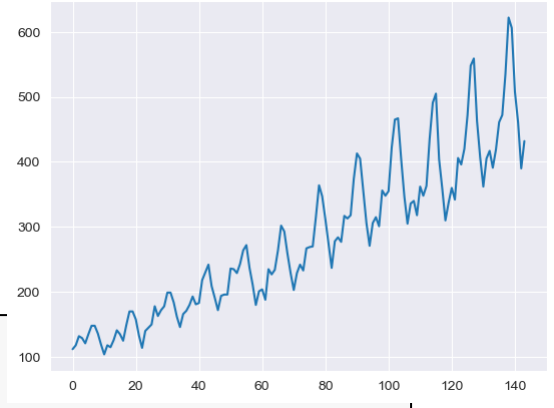



- How we figure out whether data is stationary/non-stationary?
  - Visual inspection
  - Summary statistics
  - Statistical tests (e.g., KPSS, ADF)
  - Autocorrelation Function (ACF)



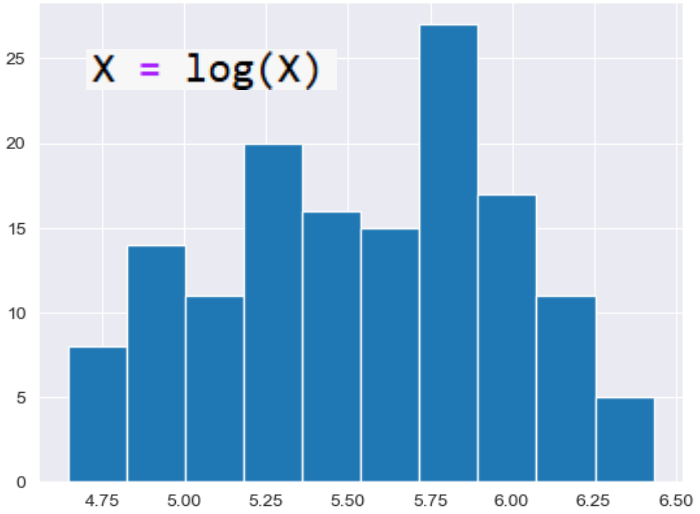
# Summary statistics

- A quick and dirty check
- You can split data into two/more partitions and compare the mean and variance of each group.
- If they differ statistically significantly, data is likely non-stationary.



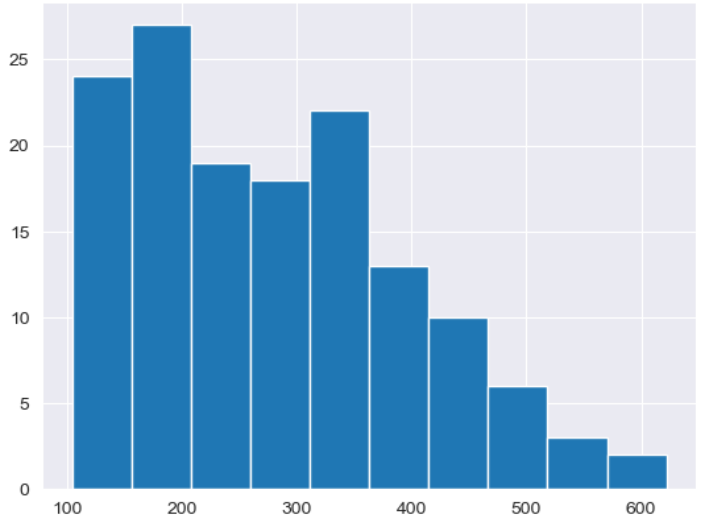

```
df.hist()
pyplot.show()

X = df.values
split = round(len(X) / 2)
X1, X2 = X[0:split], X[split:]
mean1, mean2 = X1.mean(), X2.mean()
var1, var2 = X1.var(), X2.var()
print('mean1=%f, mean2=%f' % (mean1, mean2))
print('variance1=%f, variance2=%f' % (var1, var2))
```




```
mean1=5.175146, mean2=5.909206
variance1=0.068375, variance2=0.049264
```

We have a stationary time series




```
mean1=182.902778, mean2=377.694444
variance1=2244.087770, variance2=7367.962191
```

We have a non-stationary time series (Maybe!)

# Statistical Tests

- Statistical tests (unit root test) make strong assumptions about your data. They can only be used to inform the degree to which a null hypothesis can be rejected or fail to be reject:
  - Augmented Dickey-Fuller (ADF) test
  - Kwiatkowski-Phillips-Schmidt-Shin (KPSS) Test

In ADF test:

- *Null Hypothesis (H0):* If failed to be rejected, suggests the time series has a unit root, meaning it is non-stationary. It has some time-dependent structure.
- *Alternate Hypothesis (H1):* The null hypothesis is rejected; suggests time series doesn't have a unit root and is stationary (doesn't have time-dependent structure).

Interpret this result using p-value and a threshold (such as 5% or 1%).
p-value <= 0.05: Reject the null hypothesis (stationary)
p-value > 0.05: Fail to reject the null hypothesis (non-stationary)

In KPSS test:

- *Null Hypothesis ($H_0$): The time series is stationary.*
- *Alternative Hypothesis ($H_1$): The time series has a unit root and is non-stationary*

Interpret this result using the p-value and a threshold (such as 5% or 1%).
p-value > 0.05: Fail to reject the null hypothesis (stationary)
p-value <= 0.05: Reject the null hypothesis (non-stationary)

# Using ADF and KPSS Together

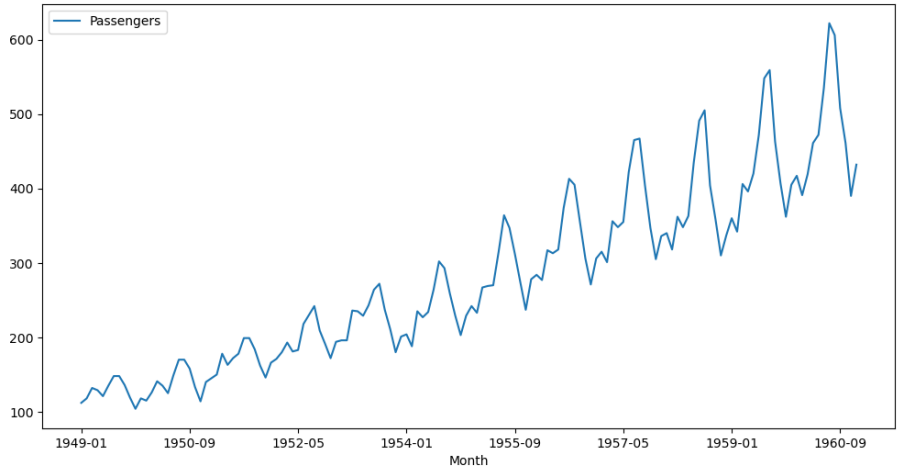
- The two tests have opposite null hypotheses, so they are complementary — use both.
  - ADF: $H_0$ = the series has a unit root **(non-stationary)**
  - KPSS: $H_0$ = the series is stationary **(stationary)**

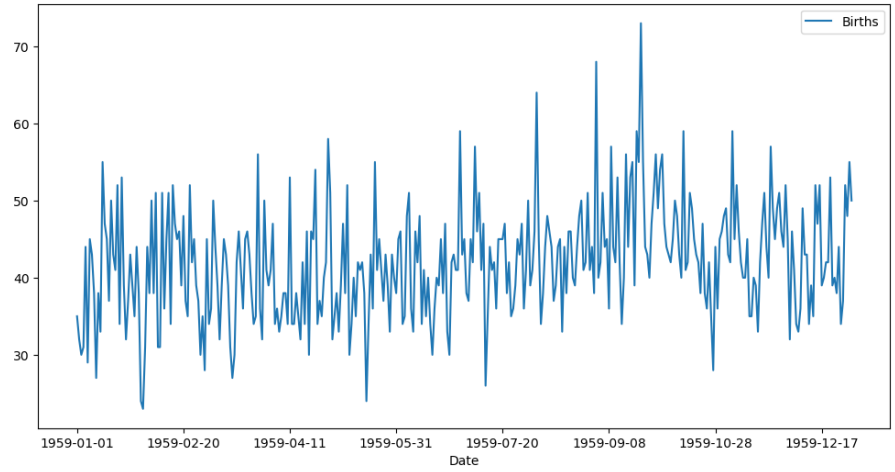
| ADF result | KPSS result | Conclusion |
|---|---|---|
| Reject $H_0$ (stationary) | Fail to reject $H_0$ (stationary) | Stationary |
| Fail to reject $H_0$ | Reject $H_0$ | Non-stationary — difference it |
| Fail to reject $H_0$ | Fail to reject $H_0$ | Not enough evidence — need more data |
| Reject $H_0$ | Reject $H_0$ | Trend-stationary — de-trend, then re-test |

- A small p-value means "reject $H_0$" in both tests — but because the nulls are opposite, it means the opposite conclusion in each.

# Statistical Tests

```
from statsmodels.tsa.stattools import adfuller
X = df.values
result = adfuller(X)
print('ADF Statistic: %f' % result[0])
print('p-value: %f' % result[1])
```




```
ADF Statistic: 0.815369
p-value: 0.991880
```

```
ADF Statistic: -4.808291
p-value: 0.000052
```

Fail to reject null hypothesis (H0), data is non-stationary

Reject null hypothesis (H0), data is stationary

# Autocorrelation/Partial Autocorrelation Function (ACF/PACF)

- Autocorrelation analysis is important step in Exploratory Data Analysis (EDA)
- Autocorrelation analysis helps in detecting hidden patterns and seasonality, and in checking for randomness.
- It is especially important when you intend to use an ARIMA model for forecasting because the autocorrelation analysis helps to identify the orders for the ARIMA model.

- Autocorrelation Function (ACF): correlation between a time series and its own past values at different time lags. (ACF at lag $k$ is the correlation between the time series values at time $t$ and the values at time $t-k$).
- Partial Autocorrelation Function (PACF): direct relationship between an observation and its lag, excluding the influence of other lags (PACF at lag $k$ is correlation between time series values at time $t$ and values at time $t-k$, after removing the effect of correlations at all lags less than $k$).

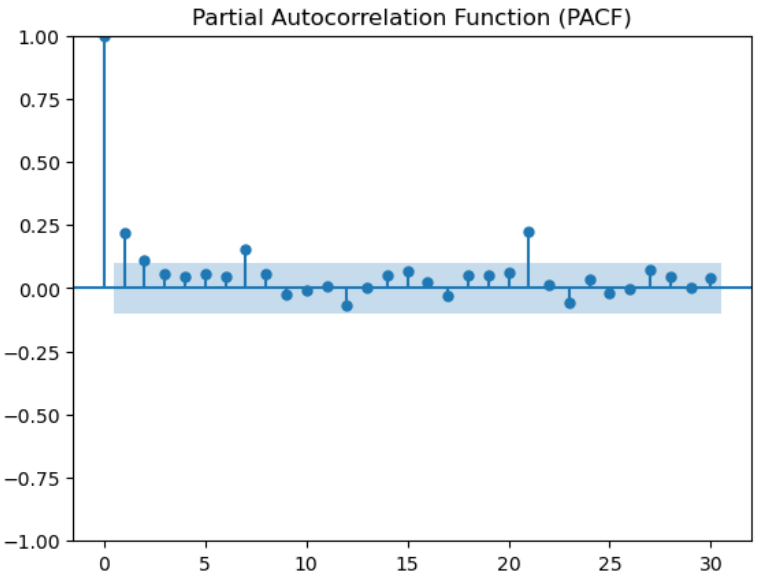
You can use ACF/PACF to determine if your data is stationary.

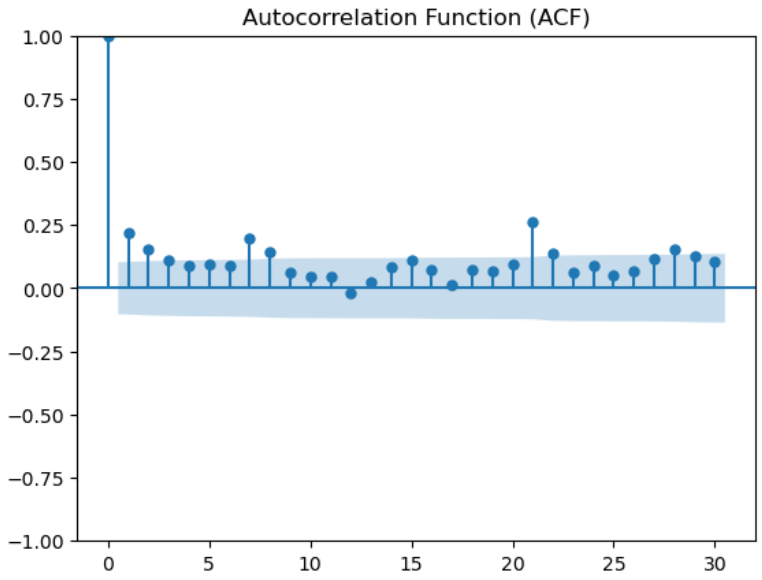
- Stationary data:
  - ACF plot: autocorrelation values quickly drop to zero after a few lags.
  - PACF plot: has a few significant spikes at the beginning and then quickly cuts off to zero.
- Non-stationary data:
  - ACF plot: autocorrelation values decay slowly and continue to be significant for many lags.
  - PACF plot: may have several significant spikes or show a gradual decay, not an abrupt cut-off.

# Plotting ACF/PACF

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
# Plotting the Autocorrelation Function (ACF)
plt.figure(figsize=(12, 6))
plot_acf(df['Births'], lags=30, alpha=0.05)  # 30 lags
plt.title('Autocorrelation Function (ACF)')
plt.show()

# Plotting the Partial Autocorrelation Function (PACF)
plt.figure(figsize=(12, 6))
plot_pacf(df['Births'], lags=30, alpha=0.05, method='ywm')
plt.title('Partial Autocorrelation Function (PACF)')
plt.show()
```



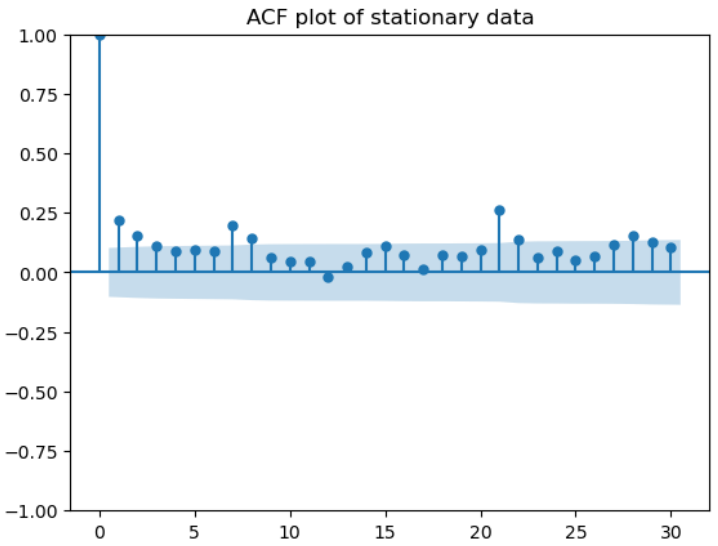



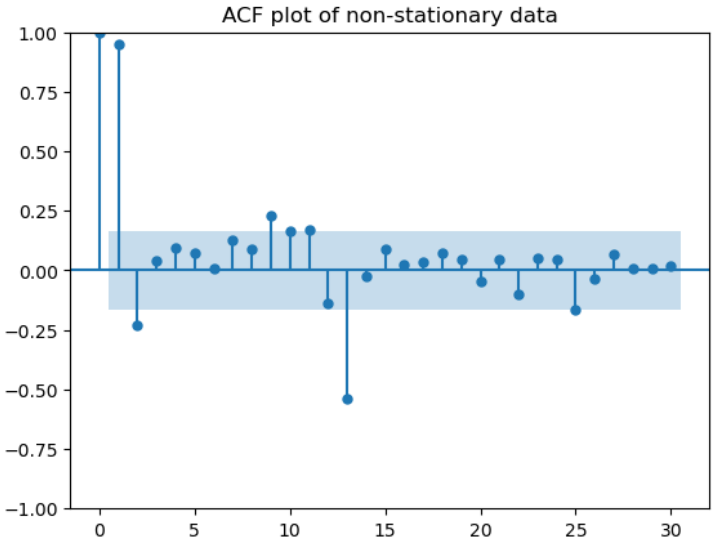

Blue region: 5% significance threshold/95% confidence band

# ACF/PACF to Check Stationary

```python
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
# Plotting the Autocorrelation Function (ACF)
plt.figure(figsize=(12, 6))
plot_acf(df1['Births'], lags=30, alpha=0.05)  # 30 lags
plt.title('ACF plot of stationary data')
plt.show()

# Plotting the Partial Autocorrelation Function (PACF)
plt.figure(figsize=(12, 6))
plot_pacf(df2['Passengers'], lags=30, alpha=0.05)
plt.title('ACF plot of non-stationary data')
plt.show()
```




*Note*: Not always the best approach to check for stationary/non-stationary!

# How to Make Time Series Stationary?

- Eliminating/reducing trend and seasonality:
  - Transformations such as logarithms (can help to stabilize the variance of a time series).
  - Differencing (can help stabilize the mean of time series by removing changes in the level of a time series). $y_t - y_{t-1}$

We can apply:

- Lag-2/Second-order differencing:

Occasionally the differenced data will not appear to be stationary and it may be necessary to difference the data a second time to obtain a stationary series (going beyond second orders are typically unnecessary)

$$y_t - y_{t-1} - (y_{t-1} - y_{t-2}) = y_t - 2y_{t-1} + y_{t-2}$$

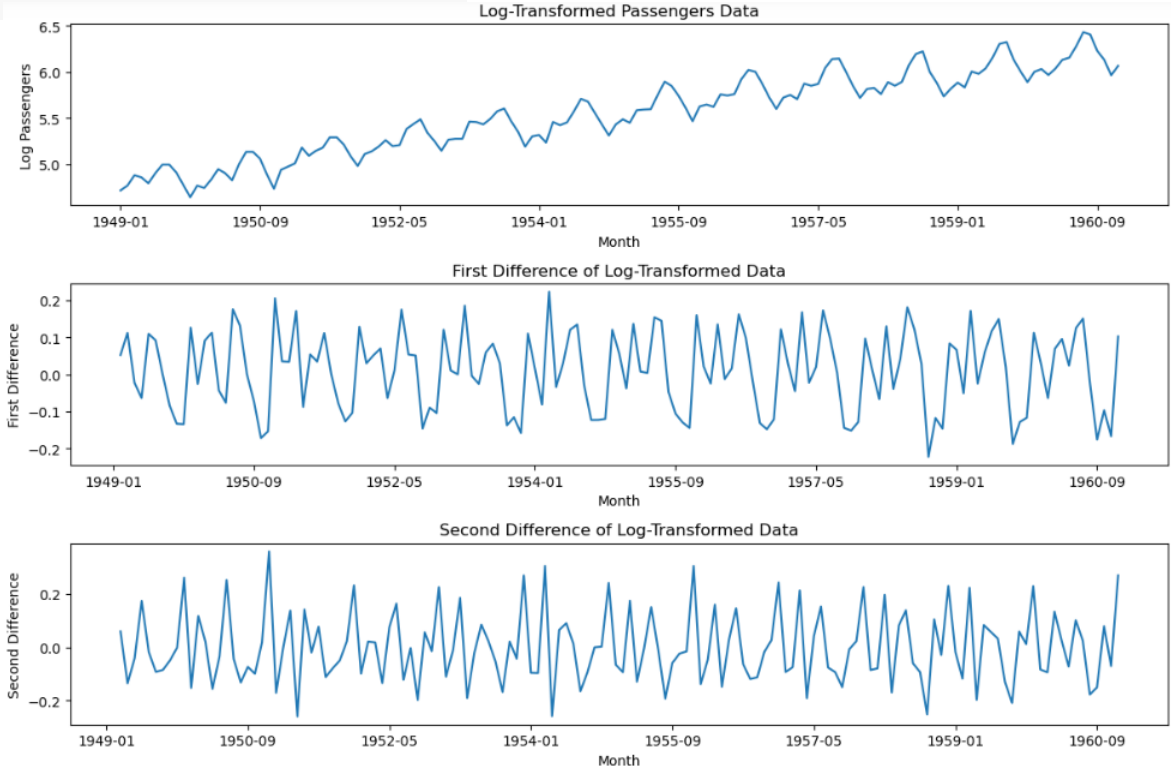
- Lag-m/Seasonal differencing (difference between an observation and the previous observation from the same season)

$$y_t - y_{t-m} \ , \qquad m = \#of\ seasons$$

# How to Make Time Series Stationary?

```
# Apply log transform to the 'Passengers' column
df['Log_Passengers'] = np.log(df['Passengers'])
# Apply first differencing to the log-transformed data
df['First_Diff'] = df['Log_Passengers'] - df['Log_Passengers'].shift(1)
# Apply second differencing to the first differenced data
df['Second_Diff'] = df['First_Diff'] - df['First_Diff'].shift(1)

# Plotting
plt.figure(figsize=(12, 8))
plt.subplot(3, 1, 1)
df['Log_Passengers'].plot(title='Log-Transformed Passengers Data')
plt.ylabel('Log Passengers')
plt.subplot(3, 1, 2)
df['First_Diff'].plot(title='First Difference of Log-Transformed Data')
plt.ylabel('First Difference')
plt.subplot(3, 1, 3)
df['Second_Diff'].plot(title='Second Difference of Log-Transformed Data')
plt.ylabel('Second Difference')
plt.tight_layout()
plt.show()
```



# 2.4: Statistical Models for time-series Forecasting

# Model-based Prediction - ARIMA

- AutoRegressive Integrated Moving Average (ARIMA)
  - "AutoRegressive"
  - "Integrated" (the reverse of differencing)
  - "Moving Average"

  ARIMA model is a class of linear models that utilizes historical values to forecast future values.

# Autoregressive Models

- *Recap:* In a multiple regression model, we forecast the variable of interest using a linear combination of predictors.
- In an autoregression model, we forecast the variable of interest using a linear combination of past values of the variable.
- The AR model assumes that the current value ($y_t$)is dependent on previous values ($y_{t-1}, y_{t-2}, y_{t-3}$, …). Because of this assumption, we can build a linear regression model.

- The term autoregression indicates that it is a regression of the variable against itself.

- AR(p) : An autoregressive model of order $p$:

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p\, y'_{t-p} + \varepsilon_t$$

White Noise

- Autoregressive models are remarkably flexible at handling a wide range of different time series patterns.

# AR Order

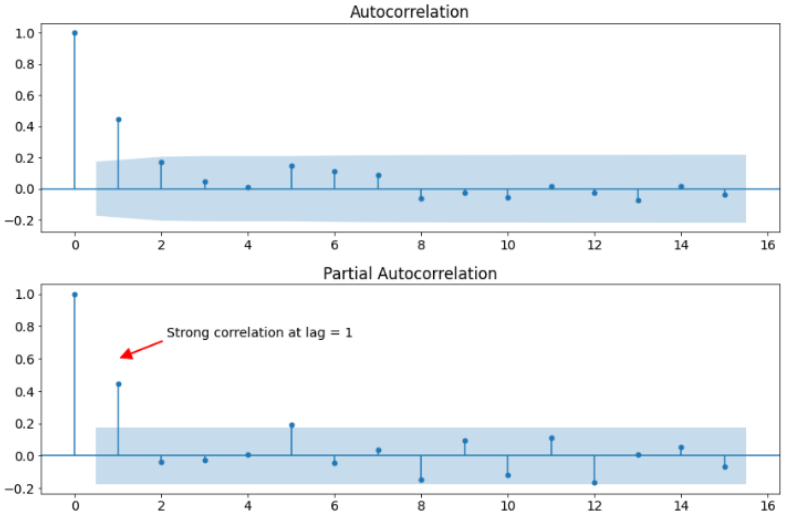
- To figure out the order of an AR model, you would use the PACF (The PACF of an AR process should cut off to zero beyond the order p)

Damped exponential and/or sine function

| | AR($p$) | MA($q$) | ARMA($p$, $q$) |
|---|---|---|---|
| ACF | Tails off (Geometric decay) | Significant at lag $q$ / Cuts off after lag $q$ | Tails off (Geometric decay) |
| PACF | Significant at each lag $p$ / Cuts off after lag $p$ | Tails off (Geometric decay) | Tails off (Geometric decay) |

Damped exponential and/or sine function

- Example: AR(1)



# Moving Average Models

- Rather than using past values of the forecast variable in a regression, a moving average model uses past forecast errors in a regression-like model.
- The MA model assumes that the current value ($y_t$) is **dependent on the error terms** including the current error ($\varepsilon_t, \varepsilon_{t-1}, \varepsilon_{t-2}, \varepsilon_{t-3}, \ldots$)
- MA(q) : A moving average model of order q:

$$y_t = c + \varepsilon_t + \theta_1\varepsilon_{t-1} + \theta_2\varepsilon_{t-2} + \cdots + \theta_q\varepsilon_{t-q}$$

White Noise

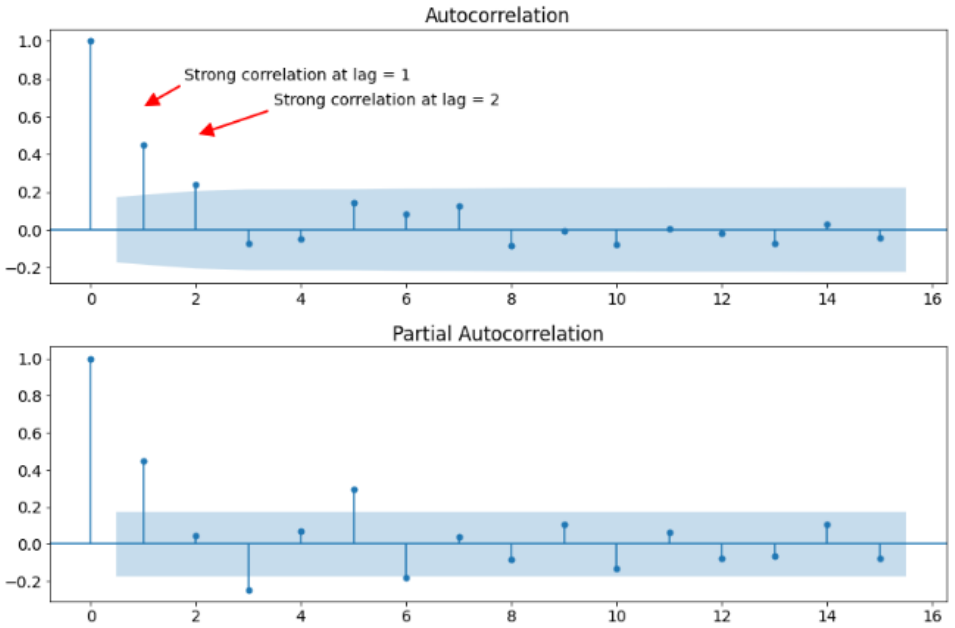
- Each value of $y_t$ can be thought of as a weighted moving average of the past few forecast errors

# MA Order

- To figure out the order of an MA model, you would use the ACF
- Unlike an autoregressive process, which has a slowly decaying ACF, the definition of the MA process ensures a sharp cutoff of the
- ACF for any value greater than q

| | AR($p$) | MA($q$) | ARMA($p$, $q$) |
|---|---|---|---|
| ACF | Tails off (Geometric decay) | Significant at lag $q$ / Cuts off after lag $q$ | Tails off (Geometric decay) |
| PACF | Significant at each lag $p$ / Cuts off after lag $p$ | Tails off (Geometric decay) | Tails off (Geometric decay) |

- Example: MA(2)



# AutoRegressive Integrated Moving Average (ARIMA)

- *If we combine differencing with autoregression and a moving average model, we obtain ARIMA model.*

- $ARIMA(p, d, q)$: An *ARIMA model of order* $p, d, q$:

$$y'_t = c + \phi_1 y'_{t-1} + \cdots + \phi_p y'_{t-p} + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t$$

$$y'_t = c + \sum_{n=1}^{p} \phi_n y'_{t-n} + \sum_{n=1}^{q} \theta_n \varepsilon_{t-n} + \varepsilon_t$$

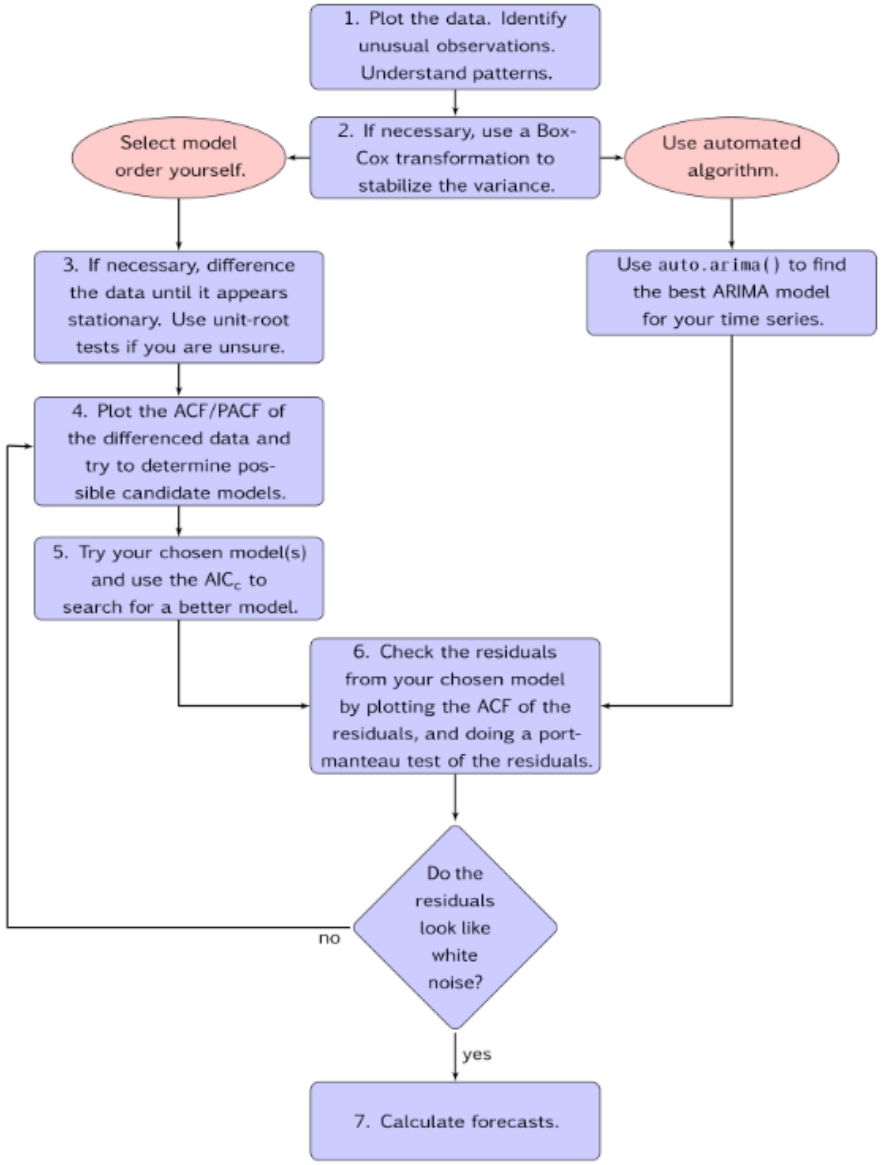
  - $y'_t$ : *is a differenced series (it may have been differenced more than once).*
  - $p$: *order of autoregressive (the number of lag observations incorporated in the mode)*
  - $d$: *degree of differencing involved (denoting the number of times raw observations undergo differencing)*
  - $q$: *order of the moving average (indicating the size of the moving average window)*
  - $\varepsilon$: *forecast error terms, the moving-average components*
  - $\phi$: *coefficients of the autoregressive components (lags)*
  - $\theta$: *coefficients of the lagged forecast errors*

- Maximum likelihood estimation (MLE)
  - Once the model order has been identified, we need to estimate the parameters $(c, \phi_1, \ldots, \phi_p, \theta_1, \ldots, \theta_q)$ using MLE
  - Finds the values of the parameters which maximize the probability of obtaining the data that we have observed
  - Similar to the least squares estimates that would be obtained by minimizing

$$\sum_{t=1}^{T} \varepsilon_t^2$$

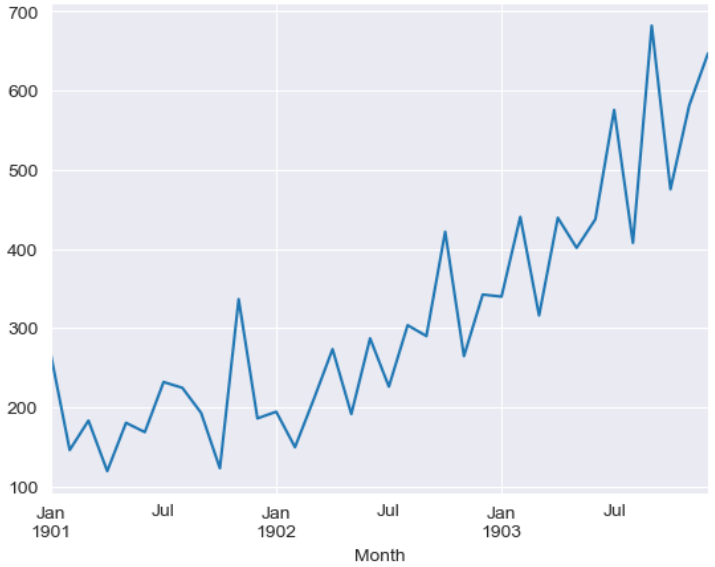
# ARIMA algorithm

- When fitting an ARIMA model to a set of (non-seasonal) time series data, the following procedure should be followed:
  - Plot the data and identify any unusual observations.
  - If necessary, transform the data to stabilize the variance.
  - If the data are non-stationary, take first differences of the data until the data are stationary.
  - Determine the order of ARIMA
  - Check the residuals from your chosen model by plotting the ACF of the residuals. If they do not look like white noise, try a modified model.
  - Once the residuals look like white noise, calculate forecasts.



# ARIMA Example

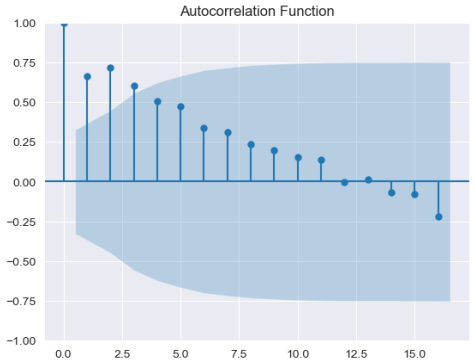
```
Month
1901-01-01    266.0
1901-02-01    145.9
1901-03-01    183.1
1901-04-01    119.3
1901-05-01    180.3
Name: Sales, dtype: float64
```

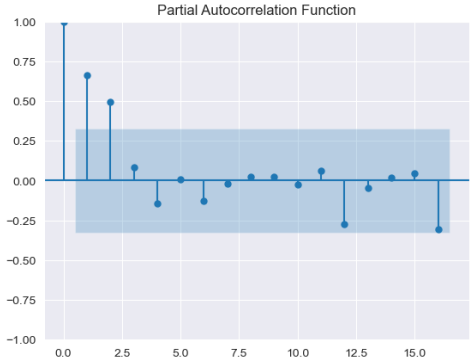



```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# Autocorrelation plot
plot_acf(series)
plt.title('Autocorrelation Function')
plt.show()

# Partial Autocorrelation plot
plot_pacf(series)
plt.title('Partial Autocorrelation Function')
plt.show()
```

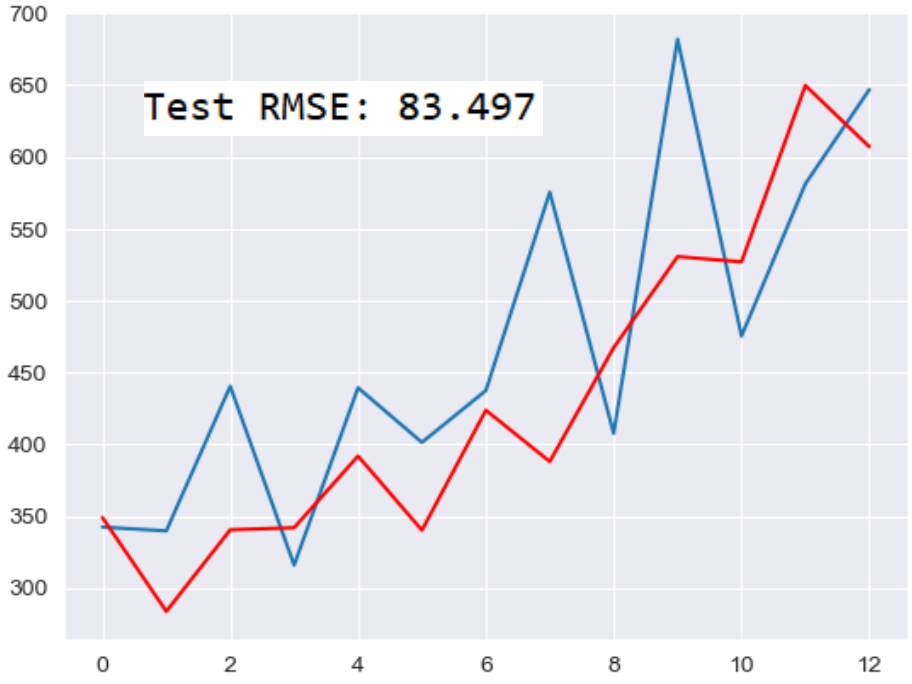


# ARIMA Example

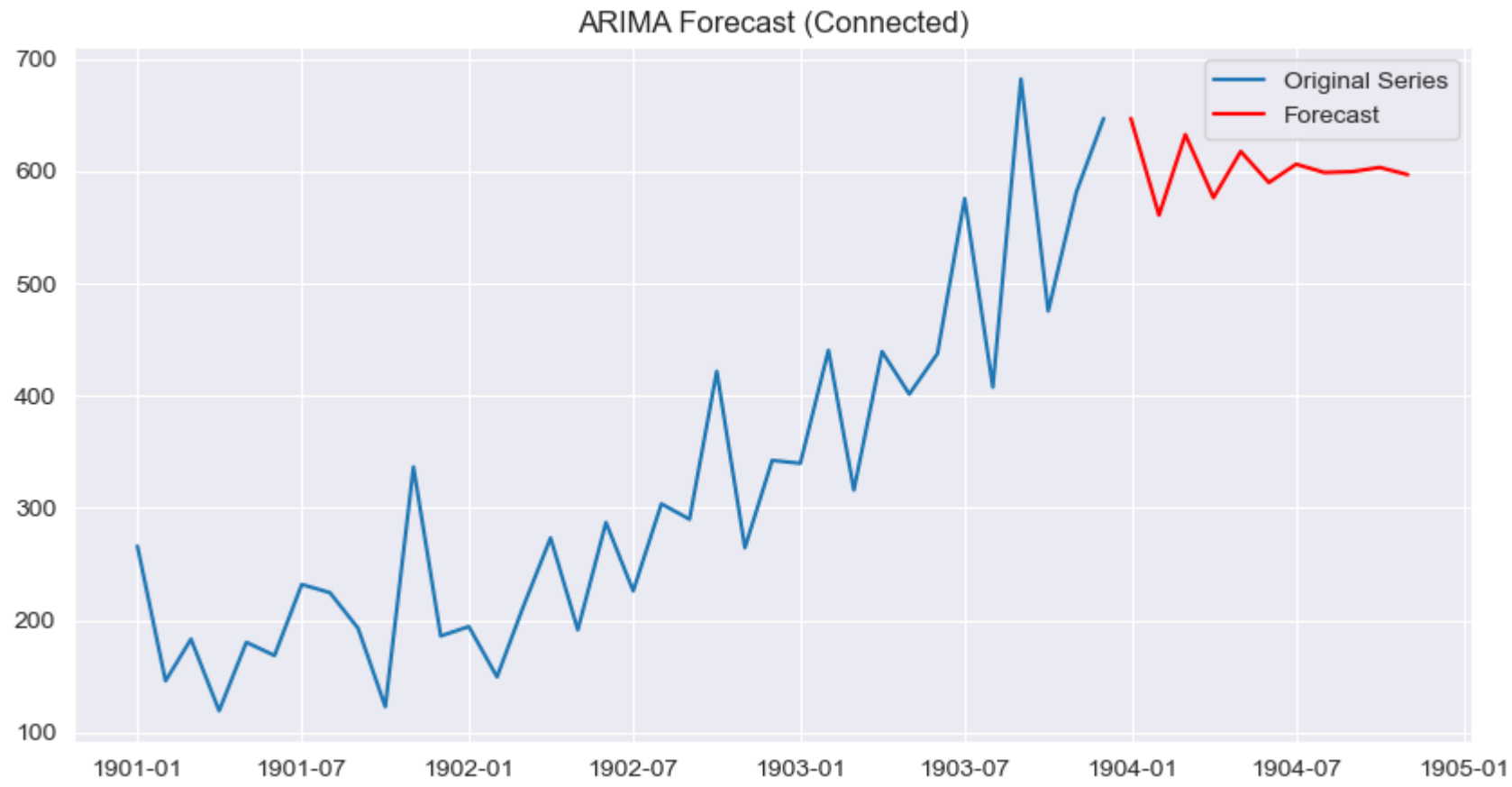
```
X = series.values
size = int(len(X) * 0.66)
train, test = X[0:size], X[size:len(X)]
history = [x for x in train]
predictions = list()
# walk-forward validation
for t in range(len(test)):
    model = ARIMA(history, order=(2,1,2))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = test[t]
    history.append(obs)
    print('predicted=%f, expected=%f' % (yhat, obs))
# Print model summary
print(model_fit.summary())
# evaluate forecasts
rmse = sqrt(mean_squared_error(test, predictions))
print('Test RMSE: %.3f' % rmse)
# plot forecasts against actual outcomes
pyplot.plot(test)
pyplot.plot(predictions, color='red')
pyplot.show()
```

```
==============================================================================
Dep. Variable:                      y   No. Observations:                   35
Model:                 ARIMA(2, 1, 2)   Log Likelihood                -191.382
Date:                Fri, 12 Sep 2025   AIC                            392.764
Time:                        05:02:49   BIC                            400.396
Sample:                             0   HQIC                           395.367
                                  - 35
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.0927      0.355      0.261      0.794      -0.604       0.789
ar.L2         -0.0135      0.235     -0.057      0.954      -0.475       0.448
ma.L1         -1.3048     14.617     -0.089      0.929     -29.954      27.345
ma.L2          0.9986     22.357      0.045      0.964     -42.821      44.818
sigma2      3804.1962   8.48e+04      0.045      0.964   -1.62e+05    1.7e+05
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 0.05
Prob(Q):                              0.92   Prob(JB):                         0.98
Heteroskedasticity (H):               1.00   Skew:                             0.03
Prob(H) (two-sided):                  1.00   Kurtosis:                         2.83
===================================================================================
```



# ARIMA Example

```
# Forecast next 10 months
forecast = model_fit.forecast(steps=10)
print("Forecasted values:")
print(forecast)
```



# Seasonal-ARIMA (SARIMA)

- The integrated element refers to differencing, allowing ARIMA to support time series data with a trend.
- Although ARIMA can handle data with a trend, it does not support time series with a seasonal component.
- ARIMA expects data that is either not seasonal or has the seasonal component removed, e.g. seasonal differencing.

- SARIMA is Seasonal-ARIMA and includes seasonality contribution to the forecast.
- The Autoregressive (AR), Integrated (I), and Moving Average (MA) parts of the model remain as that of ARIMA. The addition of Seasonality adds robustness to the SARIMA model.

- It has three new hyperparameters to specify the autoregression (AR), differencing (I) and moving average (MA) for the seasonal component of the series, as well as an additional parameter for the period of the seasonality.

$$SARIMA\ \underbrace{(p,d,q)}_{\text{Non-seasonal part of model}}\ \underbrace{(P,D,Q)m}_{\text{Seasonal part of model}}$$

# Seasonal-ARIMA (SARIMA)

- It has three new hyperparameters to specify the autoregression (AR), differencing (I) and moving average (MA) for the seasonal component of the series, as well as an additional parameter for the period of the seasonality.

$$SARIMA(p, d, q)(P, D, Q)m$$

  - $p$: *Trend autoregressive order*
  - $d$: *Trend differencing degree*
  - $q$: *Trend moving average order*

  - P: *Seasonal autoregressive order ($P = 1$ uses the first seasonally offset observation, e.g. $t - (m * 1)$, Or a $P = 2$, uses the last two seasonally offset observations $t - (m * 1), t - (m * 2)$)*
  - $D$: *Seasonal difference order ($D = 1$ would calculate a first order seasonal difference)*
  - $Q$: *Seasonal moving average order ($Q = 1$ would use a first order errors)*
  - $m$: *The number of time steps for a single seasonal period (m parameter influences the $P$, $D$, and $Q$ parameters)*

$$y'_t = c + \sum_{n=1}^{p} \phi_n \, y'_{t-n} + \sum_{n=1}^{q} \theta_n \varepsilon_{t-n} + \sum_{n=1}^{P} \eta_n \, y'_{t-mn} + \sum_{n=1}^{Q} \omega_n \varepsilon_{t-mn} + \varepsilon_t$$

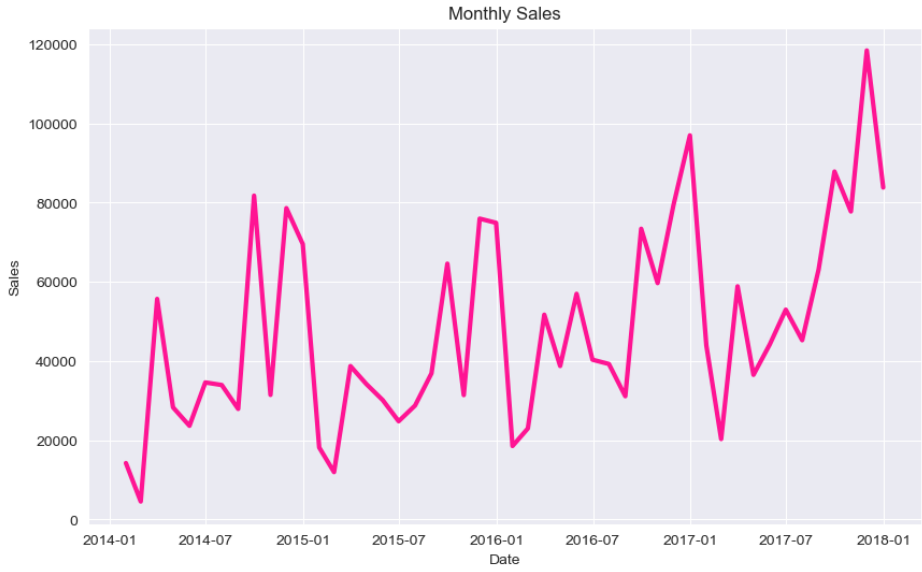
# SARIMA Example

- SARIMA:

$$y'_t = c + \sum_{n=1}^{p} \phi_n \, y'_{t-n} + \sum_{n=1}^{q} \theta_n \varepsilon_{t-n} + \sum_{n=1}^{P} \eta_n \, y'_{t-mn} + \sum_{n=1}^{Q} \omega_n \varepsilon_{t-mn} + \varepsilon_t$$

  - $y'$: *differenced time series, through both regular, d, and seasonal, D, differencing*
  - $P$: *number of seasonal auto-regressors*
  - $\omega$: *coefficients of the seasonal autoregressive components*
  - $Q$: *number of seasonal moving-average components*
  - $\eta$: *coefficients of the seasonal forecast errors*
  - $m$: *length of season*

| | Order Date | Sales |
|---|---|---|
| **0** | 2016-11-08 | 261.9600 |
| **1** | 2016-11-08 | 731.9400 |
| **2** | 2016-06-12 | 14.6200 |
| **3** | 2015-10-11 | 957.5775 |
| **4** | 2015-10-11 | 22.3680 |

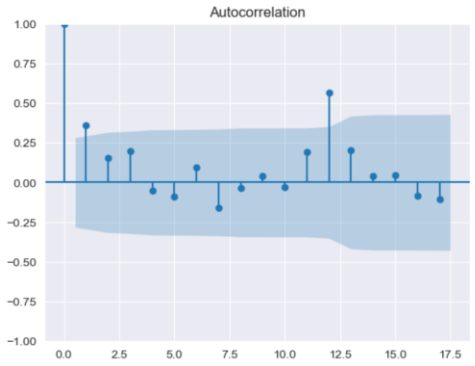


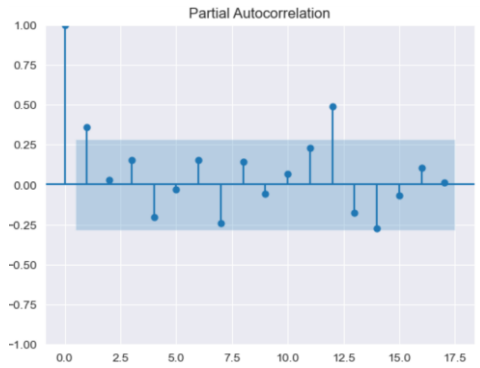
# SARIMA Example

- Check stationary

```
ADF Statistic: -4.493767844002666
p-value: 0.00020180198458237704
Stationary
```

- Find orders

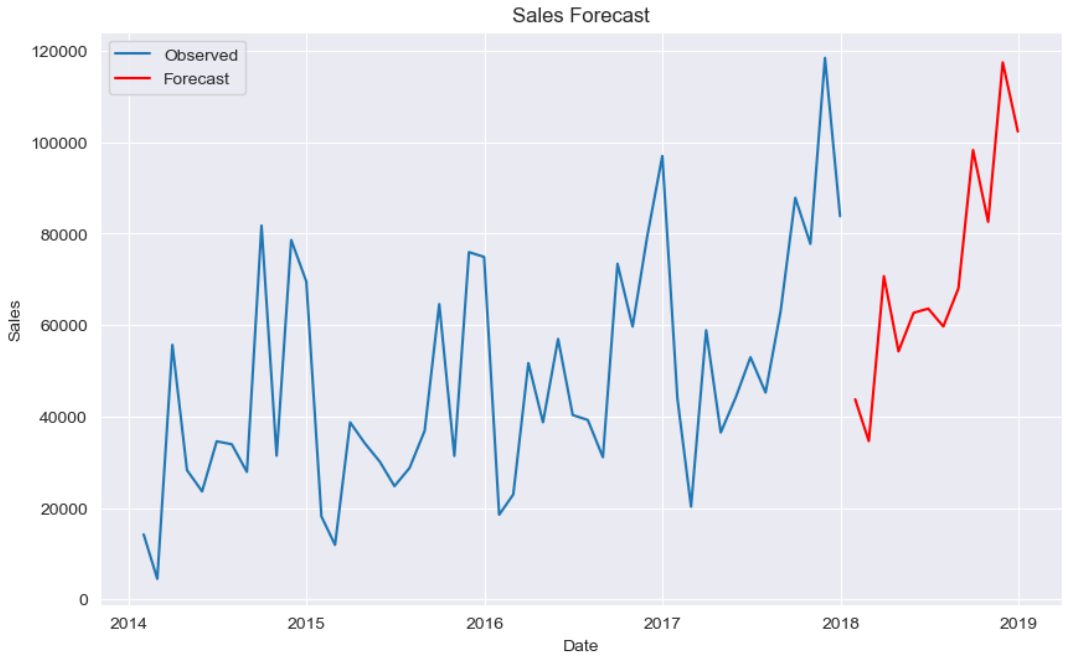



```python
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error
import matplotlib.pyplot as plt

# Fit SARIMA model
p, d, q = 1, 1, 1
P, D, Q, s = 1, 1, 1, 12
model = SARIMAX(monthly_sales, order=(p, d, q), seasonal_order=(P, D, Q, s))
results = model.fit()
# Forecast
forecast_periods = 12
forecast = results.forecast(steps=forecast_periods)
# Plot
plt.figure(figsize=(10, 6))
plt.plot(monthly_sales, label='Observed')
plt.plot(forecast, label='Forecast', color='red')
plt.title("Sales Forecast")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.show()
# Evaluate
observed = monthly_sales[-forecast_periods:]
mae = mean_absolute_error(observed, forecast)
mse = mean_squared_error(observed, forecast)
print(f'MAE: {mae}')
print(f'MSE: {mse}')
```

```
MAE: 10611.591995573883
MSE: 151953342.40275076
```



# Pros and Cons of Statistical Models

- Pros
  - These models are simple and transparent and can be understood clearly in terms of their parameters.
  - You can apply these models to fairly small data sets and still get good results.
  - These simple models perform extremely well, even in comparison to very
  - complicated machine learning models. So you get good performance without the danger of overfitting.

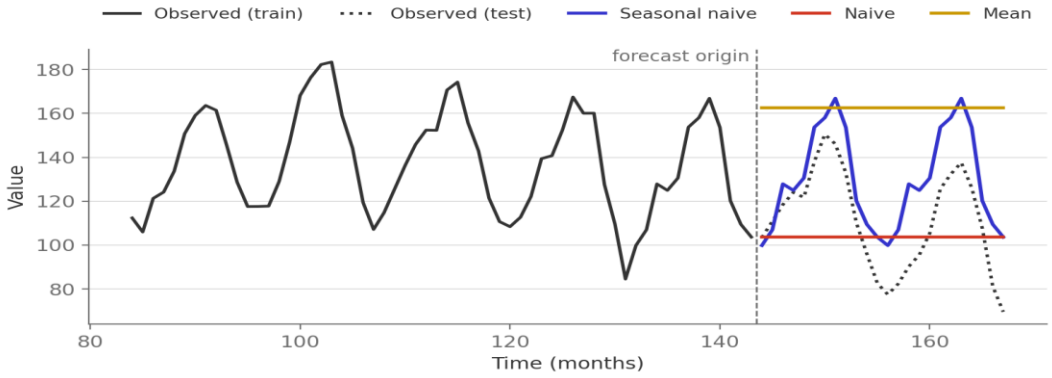
- Cons
  - Because these models are quite simple, they don't always improve performance when given large datasets. For extremely large data sets, you may do better with the complex ML models.
  - They offer only limited ways to express uncertainty relative to all the choices you make in selecting a model.
  - By definition, these models are not built to handle nonlinear dynamics and will do a poor job describing data where nonlinear relationships are dominant.

# 2.5: Evaluating and Backtesting Forecasts

- Baselines and error metrics
- Temporal splits and backtesting
- Data leakage in time series

# Baselines You Must Beat

- A baseline is a trivial forecast. Report one with every model — it is the only way to know whether the model learned anything at all.
  - Naive (random walk): $\hat{y}_{T+h|T} = y_T$
  - Seasonal naive: $\hat{y}_{T+h|T} = y_{T+h-m}$ m = seasonal period (12 monthly, 24 hourly)
  - Mean: $\hat{y}_{T+h|T} = \bar{y}$
- For many real series — demand, load, prices — the seasonal naive is very hard to beat.
- If your deep model does not beat the seasonal naive, you do not have a result — **you have a bug.**




```
# --- baselines: always compute these first ---
h, m = 24, 12                       # horizon, seasonal period
train, test = y[:-h], y[-h:]

naive    = np.repeat(train[-1], h)
seasonal = np.array([train[-m + i % m] for i in range(h)])
mean_fc  = np.repeat(train.mean(), h)
```

# Scale-dependent Error Metrics

- Forecast error on the **test** set: $e_t = y_t - \hat{y}_t$ (not residuals of training fit)
  - $\text{MAE} = (1/n)\,\Sigma_{t=1}^{n} |e_t|$ robust to outliers; the optimal forecast is the median
  - $\text{RMSE} = [\,(1/n)\,\Sigma_{t=1}^{n} e_t^2\,]^{1/2}$ penalises large errors; optimal forecast is the mean
  - $\text{ME} = (1/n)\,\Sigma_{t=1}^{n} e_t$ the bias: is the model systematically high or low?

- Both are in the units of the data — interpretable, but not comparable across series of different scale.

- Pick the one that matches the cost: if one large miss is catastrophic use RMSE; if every unit of error costs the same use MAE.

# Percentage and Scaled Errors

- $\text{MAPE} = (100/n)\,\Sigma_{t=1}^{n} |e_t / y_t|$
  - Undefined when y = 0, and it explodes when y is near zero.
  - Depends on where the error lands, not how big it is: an error of 10 on an actual of 20 scores 50%, the same error on an actual of 120 scores 8%.
  - Asymmetric: over-forecasting is punished harder. Actual 100 / forecast 150 gives 50%, but actual 150 / forecast 100 gives 33% — the same absolute error of 50.

- $\text{MASE} = \text{MAE} / \text{MAE}_{naive}$
  - where **MAE$_{naive}$** = $(1/(T-m))\,\Sigma_{t=m+1}^{T} |y_t - y_{t-m}|$ on the training set only.
  - Scale-free, defined at zero, and symmetric.
  - **MASE < 1** better than the benchmark; **MASE > 1** worse — the benchmark was the better forecast.

- For the labs and the project: report MASE (comparable) with RMSE or MAE (in units), always beside the baseline.

# Which Metric? A Comparison

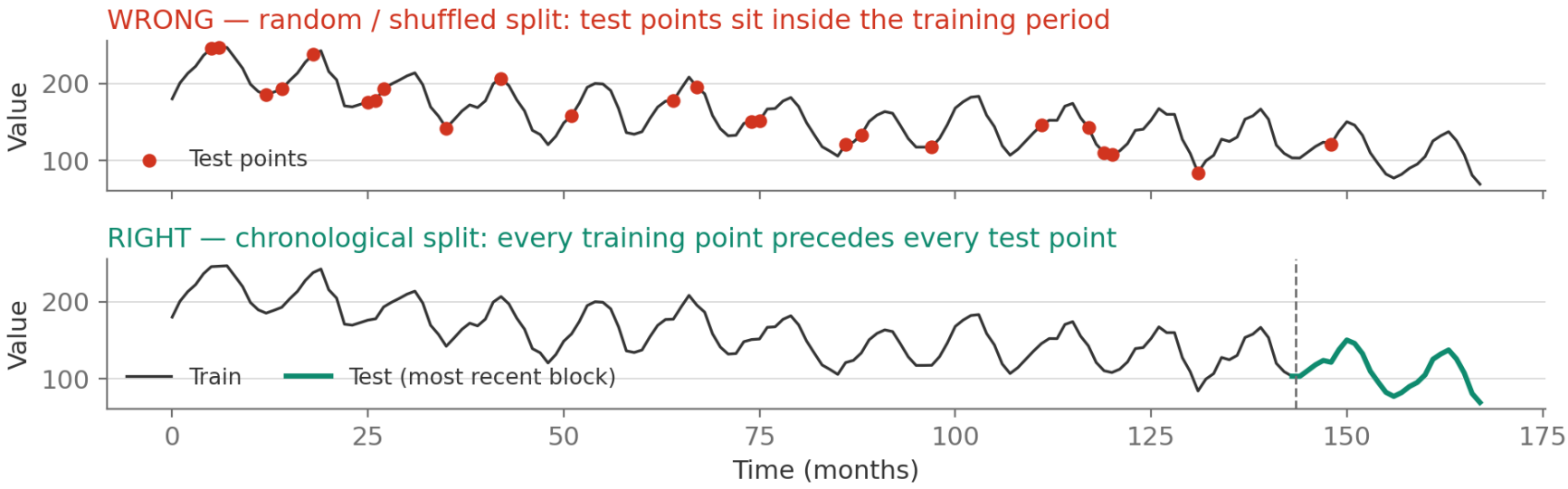
- Monthly series, 144 observations for training, the last 24 held out.

| Forecast (h = 24) | MAE | RMSE | MAPE % | MASE |
|---|---|---|---|---|
| Mean | 51.6 | 56.3 | 53.2 | 3.93 |
| Naive | 20.3 | 23.8 | 18.5 | 1.55 |
| Seasonal naive | 19.2 | 21.7 | 18.9 | 1.46 |
| **SARIMA$(1,1,1)(0,1,1)_{12}$** | **11.7** | **13.3** | **11.4** | **0.89** |

- Read the MASE column first: only the SARIMA model beats the benchmark. Mean and Naive are not just worse — they are worse than doing nothing clever.
- MAE and MAPE disagree: MAE prefers the seasonal naive over the naive (19.2 vs 20.3), MAPE prefers the naive (18.5 vs 18.9).
- Announce the metric before you see the results.
- **A MASE of 0.89 means the model cut the benchmark error by 11%.**

# Never Split a Time Series at Random

- The most common mistake in student projects — and in published papers.
  - train_test_split(..., shuffle=True) puts future observations into the training set.
  - The model interpolates between known points instead of extrapolating; the reported error can be several times too optimistic.
  - Plain k-fold cross-validation has exactly the same flaw.



# Chronological Train / Validation / Test

- Split by time, never at random:
  - **Train** — the earliest block; fit model parameters here.
  - **Validation** — the next block; choose hyper-parameters, early stopping, and model form here.
  - **Test** — the most recent block; touched exactly once, at the very end.

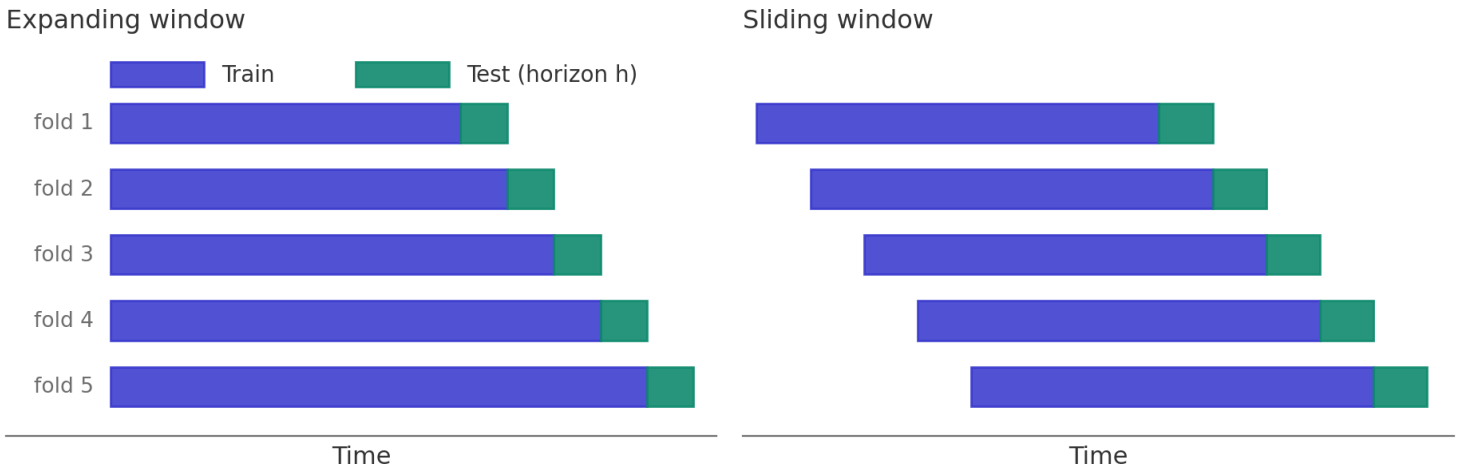
- Sizing: the test block must be at least as long as the forecast horizon h, and ideally covers two or three full seasonal cycles.

- The test set is not a development set. **Every time you look at it and change something, it silently becomes a validation set and stops being an estimate of future performance.**

- Everything learned from data is fitted on the training block only: scaling, differencing, imputation, decomposition, feature selection.

# Rolling-Origin Backtesting

- A single hold-out gives one number from one period — one draw from a noisy distribution.
- Backtesting repeats the evaluation at many forecast origins and aggregates the errors. Also called walk-forward validation, rolling-origin evaluation, or time series cross-validation.




  - **Expanding window** — the training set grows. Use when more history always helps.
  - **Sliding window** — fixed training length. Use when the process drifts or changes regime.
- Report the mean and the spread across folds, never a single number.

# Data Leakage in Time Series

- Leakage = using information at training time that would not exist at forecast time. It always inflates the reported accuracy. Check every one of these:
  - **Scaling or normalising** with the mean and standard deviation of the whole series.
  - **Imputation that looks forward** — backward-fill (NOCB) and interpolation both do (Section 2.1).
  - **Centred rolling statistics** — a window centred at t uses t+1, t+2, … Use trailing windows only.
  - **Differencing or decomposition** parameters estimated on the full series.
  - **Feature selection, tuning or early stopping** done on the test set.
  - **Overlapping windows** shared between the training and test blocks.

- Red flag: near-perfect test accuracy. **Look for leakage before you celebrate.**

# The Evaluation Protocol

- Use this in every lab and in the group project.
  - **1. Define the task** — horizon h, frequency, and what is known at forecast time.
  - **2. Split chronologically** — train / validation / test, never at random.
  - **3. Fit every preprocessing step on the training block only** — scaler, imputer, differencing.
  - **4. Compute the baselines** — naive and seasonal naive, on the same split.
  - **5. Choose the metrics before you see any result** — MASE + RMSE, and the error by horizon.
  - **6. Tune on validation** — never on test; backtest with a rolling origin.
  - **7. Evaluate on test once** — report mean and spread across folds.